

Experimental Evaluation of a Branch and Bound Algorithm for Computing Pathwidth^{*}

David Coudert^{1,2}, Dorian Mazauric³, and Nicolas Nisse^{1,2}

¹ Inria, France

² Univ. Nice Sophia Antipolis, CNRS, I3S, UMR 7271, 06900 Sophia Antipolis, France

³ Aix-Marseille Université, CNRS, LIF UMR 7279, Marseille, France

Abstract. *Path-decompositions* of graphs are an important ingredient of dynamic programming algorithms for solving efficiently many NP-hard problems. Therefore, computing the pathwidth and associated path-decomposition of graphs has both a theoretical and practical interest. In this paper, we design a Branch and Bound algorithm that computes the exact pathwidth of graphs and a corresponding path-decomposition. Our main contribution consists of several non-trivial techniques to reduce the size of the input graph (pre-processing) and to cut the exploration space during the search phase of the algorithm. We evaluate experimentally our algorithm by comparing it to existing algorithms of the literature. It appears from the simulations that our algorithm offers a significative gain with respect to previous work. In particular, it is able to compute the exact pathwidth of any graph with less than 60 nodes in a reasonable running-time (≤ 10 min.). Moreover, our algorithm achieves good performance when used as a heuristic (i.e., when returning best result found within bounded time-limit). Our algorithm is not restricted to undirected graphs since it actually computes the *vertex-separation* of di-graphs (which coincides with the pathwidth in case of undirected graphs).

1 Introduction

Because of their well known algorithmic interest, a lot of work has been devoted to the computation of *treewidth* and *tree-decompositions* of graphs [8]. On the theoretical side, exact exponential algorithms [2], Fixed Parameter Tractable (FPT) algorithms [4] and approximation algorithms have been designed. Unfortunately, most of these algorithms are impractical. For instance, the algorithm in [2] is both exponential in time and space and therefore cannot be used for graphs with more than 32 nodes. Another example is the FPT algorithms whose time-complexity is at least exponential in the treewidth: there exists no efficient implementation of the algorithm in [4] even for graphs with treewidth at most 4. On the positive side, efficient algorithms exist for computing the treewidth of particular graph classes,

^{*} This work has been partially supported by European Project FP7 EULER, ANR project Stint (ANR-13-BS02-0007), the associated Inria team AiDyNet and the project ECOS-Sud Chile.

e.g., graphs with treewidth at most 4 [10]. Many heuristics for computing lower or upper bounds on the treewidth have also been designed [5].

Surprisingly, much less work has been devoted to the computation of *pathwidth* and *path-decompositions* of graphs [8]. Indeed, the pathwidth of any n -node graph is at most $O(\log n)$ times its treewidth. Hence, providing an efficient algorithm to compute pathwidth immediately leads to an efficient approximation algorithm for computing the treewidth. In this paper, we design an algorithm that computes the pathwidth and a corresponding path-decomposition of graphs. We then provide an experimental analysis of our algorithm that presents a significant improvement with respect to existing algorithms.

1.1 Practical Computation of Pathwidth

Any *path-decomposition* of a n -node graph $G = (V, E)$ corresponds to a *layout* (i.e., an ordering) $(v_1, \dots, v_n)$ of its vertices. The *width* of a path-decomposition is the maximum size of the *border*¹ of the subgraphs induced by $\{v_1, \dots, v_i\}$, $i \leq n$. The *pathwidth* of a graph is equal to the minimum width of its path-decompositions. A motivation for computing path-decompositions comes from algorithmic applications. Indeed, given a path-decomposition of a graph, dynamic programming algorithms (whose time-complexity depends exponentially on the width of the path-decomposition) can be designed for many combinatorial optimization problems. Other motivations arise from the close relationship between pathwidth and other graph invariants (related to node layouts), e.g., the search numbers [8] and the process number [6]. Computing the pathwidth of graphs is NP-hard [8] in general. An FPT algorithm has been proposed in [4] and some kernelization reduction rules are provided in [3]. On the practical side, to the best of our knowledge, very few implementations of algorithms for computing the pathwidth (exact or bounds) have been proposed.

Exact Algorithms. Solano and Pioro proposed in [14] an exact Branch and Bound algorithm to compute the pathwidth of graphs, that checks all possible layouts of the nodes and keeps the best one. A mixed integer linear programming formulation (MILP) has been proposed in [1, 14] and a dynamic programming algorithm (exponential both in time and space) is described in [2]. None of these methods handle graphs with more than 30 nodes. As far as we know, the best practical exact algorithm for computing the pathwidth of graphs with more than 30 nodes is based on a SAT formulation of the problem that is solved using Constraint Programming solver [1].

Heuristics. A polynomial-time heuristic for vertex-separation is proposed in [6]. It aims at computing a layout of the nodes in a greedy way. At each step, the next node of the layout is chosen using a *flow circulation* method. A heuristic based on a Branch and Bound algorithm has been designed in [14]. Recently, a heuristic has been proposed that is based on the combination of a **Shake** function

¹ For any graph $G = (V, E)$ and $X \subseteq V$, the border of X is the set of nodes in X that have a neighbor in $V \setminus X$.

and **Local_Search** [9]: it consists in sequentially improving a layout by switching the nodes until a local optimum is achieved.

1.2 Contributions and Organization of the Paper

We design a Branch and Bound algorithm that computes the exact pathwidth of graphs and a corresponding path-decomposition. Basically, our algorithm explores the set of all possible layouts of the vertex-set, returning a layout with smallest width. Our main contribution consists of several non-trivial techniques to reduce the size of the input graph (pre-processing) and to cut the exploration space during the search phase of the algorithm. Note that, our algorithm is not restricted to undirected graphs since it actually computes the *vertex-separation* of digraphs (which coincides with the pathwidth in case of undirected graphs). In Section 2, we prove several technical lemmas that allow us to prove the correctness of the pre-processing phase (Section 2.2) and of the pruning procedures (Section 2.3). We present and prove our algorithm in Section 3. Finally, in Section 4, we evaluate experimentally our algorithm by comparing it to existing algorithms of the literature. It appears from the simulations that our algorithm offers a significative gain with respect to previous work. It is able to compute the exact pathwidth of any graph with less than 60 nodes in a reasonable running-time (≤ 10 min.). Moreover, our algorithm achieves good performance when used as a heuristic (i.e., when returning best result found within bounded time-limit).

2 Preliminaries

In this section, we formally define the notion of vertex-separation. Then, we give some technical lemmas that are used to prove the correctness of our algorithm.

2.1 Definitions and Notations

All graphs and digraphs considered in this paper are connected and loopless. Let $G = (V, E)$ be a graph. For any set $S \subseteq V$, let $N_G(S)$ be the set of nodes in $V \setminus S$ that have a neighbor in S . Similarly, for any digraph $D = (V, A)$ and any subset $S \subseteq V$, let $N_D^+(S)$ (resp., $N_D^-(S)$) be the set of nodes in $V \setminus S$ that have an in-neighbor (resp., an out-neighbor) in S . We omit the subscript when there is no ambiguity. For any digraph $D = (V, A)$ and $a = uv \in A$, let D/a be the graph obtained from D by contracting a . Let $D \setminus u$ be the graph obtained from D by removing u and its incident arcs.

Layouts and Vertex-Separation. Given a set S , a *layout* of S is any ordering of the elements of S . Let $\mathcal{L}(S)$ denote the set of all layouts of S . Let $S' \subseteq S$ and P, Q be two layouts of S' and $S \setminus S'$ respectively. Let $P \odot Q$ be the layout of S obtained by concatenating P and Q . Moreover, let $\mathcal{L}_P(S) = \{L \in \mathcal{L}(S) \mid L = P \odot Q, Q \in \mathcal{L}(S \setminus S')\}$, i.e., $\mathcal{L}_P(S)$ is the set of all layouts of S with prefix P .

Let $D = (V, A)$ be a digraph and let $L = (v_1, \dots, v_{|S|}) \in \mathcal{L}(S)$ be a layout of $S \subseteq V$. For any $1 \leq i \leq |S|$, let $\nu(L, i) = |N_D^+(\{v_1, \dots, v_i\})|$ and $\nu(L) =$

$\max_{i \leq |S|} \nu(L, i)$. The *vertex-separation* of D , denoted by $vs(D)$, is equal to the minimum $\nu(L)$ among all layouts L of V , i.e., $vs(D) = \min_{L \in \mathcal{L}(V)} \nu(L)$.

A digraph $D = (V, A)$ is *symmetric* if for each arc $uv \in A$, there is $vu \in A$. For any undirected graph G , let $vs(G)$ denote the vertex-separation of the corresponding symmetric digraph obtained from G by replacing each edge $\{u, v\}$ by two arcs uv and vu . It is well known that the vertex-separation of a undirected graph G equals its pathwidth $pw(G)$ [8].

2.2 Technical Lemmas for Preprocessing

In this section, we prove few technical lemmas that will be useful to prove the correctness of the preprocessing part of our algorithm. Due to lack of space, the proofs are omitted and can be found in [7]. Note that Lemma 2 extends to digraphs a known result in undirected graphs.

Lemma 1. *[Folklore] Let D be any connected digraph and let $SCC(D)$ be the set of strongly connected components of D . Then, $vs(D) = \max_{D' \in SCC(D)} vs(D')$.*

We show that we can contract some well chosen arcs of a digraph without modifying its vertex-separation. It is well known that, in undirected graphs, edge-contraction cannot increase the pathwidth. However, this is not true anymore in digraphs as shown in the following example. Let $D = (\{a, b, c, d\}, \{ab, cb, cd, da\})$. D is acyclic and therefore, $vs(D) = 0$. On the other hand, D/cb is a directed cycle with 3 nodes and $vs(D/cb) = 1$. We also show that under some conditions, the vertex-separation does not decrease after arc-contraction. Altogether, we get:

Theorem 1. *Let $D = (V, A)$ be a n -node digraph and $uv \in A$ such that, $vu \notin A$, and either $(N^-(v) = \{u\} \text{ and } N^+(u) \cap N^+(v) = \emptyset)$ or $(N^+(u) = \{v\} \text{ and } N^-(u) \cap N^-(v) = \emptyset)$. Then, $vs(D) = vs(D/uv)$. Moreover, an optimal layout for D can be obtained in linear time from an optimal layout of D/uv .*

Lemma 2. *Let $D = (V, A)$ be a n -node digraph and let $a, b, c \in V$ be three nodes with $N_D^+(b) = N_D^-(b) = \{a, c\}$, $N_D^+(a) = N_D^-(a) = \{b, x\}$, $N_D^+(c) = N_D^-(c) = \{b, y\}$, and $x \neq c$. Then, $vs(D) = vs(D/bc)$.*

2.3 Technical Lemmas for the Pruning Part

In this section, we prove few technical lemmas that will be useful to prove the correctness of the *Pruning* part of our *Branch & Bound* algorithm.

When looking for a good layout of the nodes of a digraph, our algorithm will, at some step, consider a layout P of some subset $S \subset V$ and look for the best layout L of V starting with P . Next lemma gives some conditions on a node $v \in V \setminus S$ to ensure that the best solution starting with $P \odot v$ is as good as L .

Lemma 3. *Let $D = (V, A)$ be a n -node digraph, $S \subset V$, and $P \in \mathcal{L}(S)$. If there exists $v \in V \setminus S$ such that either $N^+(v) \subseteq (S \cup N^+(S))$, or $v \in N^+(S)$ and $N^+(v) \setminus (S \cup N^+(S)) = \{w\}$. Then, $\min_{L \in \mathcal{L}_P(V)} \nu(L) = \min_{L \in \mathcal{L}_{P \odot \{v\}}(V)} \nu(L)$.*

Let L be the best layout of V having a prefix P . We express some conditions under which no layout of V starting with a permutation of P is better than L .

Lemma 4. *Let $D = (V, A)$ be a n -node digraph, $S \subset V$ and let $P, P' \in \mathcal{L}(S)$ be two layouts of S . If $\nu(P) < \min_{L \in \mathcal{L}_P(V)} \nu(L)$ or $\nu(P) \leq \nu(P')$, then $\min_{L \in \mathcal{L}_P(V)} \nu(L) \leq \min_{L \in \mathcal{L}_{P'}(V)} \nu(L)$.*

3 The Algorithm

In this section, we present our exact exponential-time algorithm for computing $vs(D)$ for any n -node digraph $D = (V, A)$. To ease the presentation, we assume that D is strongly connected. If it is not the case, we apply all the steps of our algorithm to each of its strongly connected components, and then use the construction presented in the proof of Lemma 1 to deduce in linear time the layout L of D such that $\nu(L) = vs(D)$.

Our algorithm is based on a *Branch & Bound* procedure that considers all possible layouts of V and keeps a layout L minimizing $\nu(L)$. Since the worst case time-complexity of such an algorithm is $O(n!)$, we first do a pre-processing to reduce the size of the input.

3.1 Pre-processing Phase

The following reduction rules are applied while it is possible. The rules that have been proposed for undirected graphs can be used only when D is a symmetric digraph. Note that, the digraph D^* obtained after applying once one of these rules is strongly connected, and that given the layout L^* such that $\nu(L^*) = vs(D^*)$, one can deduce in linear time a layout L for D such that $\nu(L) = vs(D)$.

Rule A: If there is $uv \in A$ such that, $vu \notin A$, and either $(N^-(v) = \{u\}$ and $N^+(u) \cap N^+(v) = \emptyset)$, or $(N^+(u) = \{v\}$ and $N^-(u) \cap N^-(v) = \emptyset)$, then let $D^* = D/uv$ (By Theorem 1, $vs(D^*) = vs(D)$).

Rule B: If there are three nodes $a, b, c \in V$ with $N_D^+(b) = N_D^-(b) = \{a, c\}$, $N_D^+(a) = N_D^-(a) = \{b, x\}$, $N_D^+(c) = N_D^-(c) = \{b, y\}$, and $x \neq c$, then let $D^* = D/bc$ (By Lemma 2, $vs(D^*) = vs(D)$).

Rule C: If D is symmetric, let G be the underlying undirected graph. It is shown in [3] that $vs(D^*) = vs(D)$ when D^* is obtained as follows.

C.1: If (in G) two degree-one vertices u and v share their neighbor, then $D^* = D \setminus u$.

C.2: If (in G) v, w are two vertices of degree two, and suppose x and y are the common neighbors of v and w , then $D^* = D/vx$.

Note that, each time that one of the above rules is applied, the number of nodes decreases by one and therefore, the worst case time-complexity is divided by n . When no reduction rules can be applied anymore, the algorithm *B&B* is applied as explained below.

3.2 Branch & Bound Phase

We now describe our *Branch & Bound*, that we call *B&B*. The main contribution of our work consists of the way we cut the exploration of all layouts of V . Intuitively, let $S \subset V$ and P be a layout of S that Procedure *B&B* is testing, i.e., let us consider a step when *B&B* is considering all layouts of V with prefix P . Moreover, let L_{UB} be the best layout of D obtained so far by *B&B*. We use the two following pruning rules:

1. Decide, using Lemma 4, if it is useful to explore layouts with prefix P . To this end, we maintain a table $\mathcal{P}$ that contains, among others, some subsets of V . If there is an entry in $\mathcal{P}$ for $S \subset V$ and satisfying some properties, we can decide that the best layout starting with the nodes in S cannot be better than L_{UB} and therefore, we do not test further layouts with prefix P .
2. Greedily extend the current prefix P with a vertex $v \in V \setminus S$, with $S = V(P)$, if v satisfies the conditions of Lemma 3. This allows us to restrict the exploration of all the layouts with prefix P to the layouts with prefix $P \odot v$.

As shown in the next section, these two pruning rules allow our algorithm to achieve better performances than existing ones for computing the vertex-separation of digraphs. Let us now describe the recursive procedure *B&B* (Algorithm 3 in [7]) more formally and prove its correctness. It takes as inputs:

- $D = (V, A)$, the considered digraph.
 - L_{UB} and UB such that L_{UB} is the best layout of V obtained so far and $UB = \nu(L_{UB})$. Note that UB is an upper bound for $vs(D)$.
 - A layout P of a subset $S \subseteq V$. Intuitively, P is the prefix of the layouts that will be tested by this execution of *B&B*. That is, either Procedure *B&B* will find a layout $L = P \odot Q$ of V such that $\nu(L) < \nu(L_{UB})$ or it decides that $\nu(L) \geq \nu(L_{UB})$ for any $L \in \mathcal{L}_P(V)$.
 - $\mathcal{P}$ is a set of triples $(S_i, \nu_i, b_i)_{i \leq \rho}$, where $S_i \subset V$, ν_i is an integer, and b_i is a boolean for any $i \leq \rho$. Intuitively, $(S_i, \nu_i, b_i) \in \mathcal{P}$ means that a layout P_i of S_i has already been checked, and, if $b_i = 1$, then it is useless to test any other layout of V starting with the nodes in S_i .
- Moreover, $\nu_i = \nu(P_i)$ and $b_i = 0$ if $\nu_i = \min_{L \in \mathcal{L}_{P_i}(V)} \nu(L)$.

The initial values for the inputs of *B&B* are: $P = \emptyset$, L_{UB} is any layout L of V and $UB = \nu(L) < |V|$, and $\mathcal{P} = \emptyset$. Let $\mathcal{P} = (S_i, \nu_i, b_i)_{i \leq \rho}$ and (L_{UB}, UB) be the current values of the global variables at some step of the execution of the algorithm. Then, executing *B&B*($D, P, UB, L_{UB}, \mathcal{P}$), with $P = (v_1, \dots, v_k)$ being a layout of $S = \{v_1, \dots, v_k\}$, proceeds as follows.

- if $\nu(P) \geq UB$, then *B&B*($D, P, UB, L_{UB}, \mathcal{P}$) does nothing, i.e., the exploration of the layouts starting by P stops. Indeed, $\min_{L \in \mathcal{L}_P(V)} \nu(L) \geq \nu(P)$ by definition. Therefore, $\min_{L \in \mathcal{L}_P(V)} \nu(L) \geq UB$ and no layout of V starting with P can be better than L_{UB} .

- else, if (S, x, b) belongs to $\mathcal{P}$, then this means that a layout P' of S has been already tested, $x = \nu(P')$ and $UB \leq \min_{L \in \mathcal{L}_{P'}(V)} \nu(L)$. If $b = 1$ or $\nu(P) \geq \nu(P')$, then $B\&B(D, P, UB, L_{UB}, \mathcal{P})$ does nothing, i.e., the exploration of the layouts starting by P stops. Indeed, either $x < \min_{L \in \mathcal{L}_{P'}(V)} \nu(L)$ (if $b = 1$) or $\nu(P) \geq \nu(P')$. By Lemma 4, $\min_{L \in \mathcal{L}_{P'}(V)} \nu(L) \leq \min_{L \in \mathcal{L}_P(V)} \nu(L)$. Again, no layout of V starting with P can be better than L_{UB} .
- otherwise, $B\&B(D, P, UB, L_{UB}, \mathcal{P})$ applies the sub-procedure *Greedy*(D, P) (Algorithm 1 in [7]) that returns a prefix P' extending P (i.e., P is a prefix of P') and $\min_{L \in \mathcal{L}_P(V)} \nu(L) = \min_{L \in \mathcal{L}_{P'}(V)} \nu(L)$. Then, it calls $B\&B(D, P' \odot v, UB, L_{UB}, \mathcal{P})$ for all $v \in V \setminus V(P')$ such that $\nu(P' \odot v) < UB$, starting from the most promising vertices (i.e., by increasing value of $\nu(P' \odot v)$).

At the end of the exploration of the layouts with prefix P , we update the table $\mathcal{P}$ (Algorithm 2 in [7]). That is,

- if there was no triple $(S, *, *)$ in $\mathcal{P}$, then, $(S, \nu(P), b)$ is added to $\mathcal{P}$ with $b = 0$ if and only if $\nu(P) = \nu(L_{UB})$.
- else, i.e., if there is $(S, x, 0) \in \mathcal{P}$ with $\nu(P) < x$ and $\nu(P) < UB$, then $(S, \nu(P), b)$ replaces $(S, x, 0)$ in $\mathcal{P}$ with $b = 0$ iff $\nu(P) = \nu(L_{UB})$.

4 Simulations and Interpretation of Results

In this section, we evaluate the performance of our algorithm. In particular, we compare it to other exact algorithms and heuristics of the literature. We also analyze the impact of the three optimization phases of our algorithm : pre-processing, greedy steps, and pruning using prefixes. It appears that our algorithm is able to compute the vertex-separation of all graphs with at most 60 nodes but also for some graphs up to 250 nodes. Not only does our algorithm outperform the performance of existing exact algorithms but it can also be used as a good heuristic to obtain upper bounds on the vertex-separation.

4.1 Implementation

We have implemented several variants of our Branch and Bound algorithm in order to analyze the impact of each of our optimization sub-procedures. Briefly, **BAB** is the basic version of the Branch and Bound algorithm (as already implemented in [14]). It takes as an input a digraph D and is applied sequentially on each of the strongly connected components of D . Then, in **BAB-G** we add the greedy-step process (Algorithm 1 in [7]) to **BAB**, in **BAB-GP** we add the pre-processing phase to **BAB-G**, and finally, our main algorithm is **BAB-GPP** in which we add to **BAB-GP** the pruning-process by storing some subsets of nodes a layout of which has already been tested as prefix (Algorithm 2 in [7]).

Our algorithms are implemented in Cython using the *Sage* open-source mathematical software [15]. To speed-up critical operations on neighborhoods (union, intersection, size, etc.), we store out-neighborhoods using bitsets (in particular, a single integer when $n \leq 64$). This enables us to use bitwise operations (OR for union, AND for intersection, etc.). For **BAB-GPP**, the prefixes are stored in a

tree structure offering fast access to already tested prefixes. We parameterized the maximum length of a prefix (which corresponds to the tree depth) and set it by default to $\min\{n/3, 50\}$. The total number of stored prefixes is also parameterized to limit memory usage. By default we store at most 10^6 prefixes. Finally, we used a timer to limit computation time per (di)graph. When the time is up, we return the best solution found so far and record it as an upper-bound on $vs(D)$. Following [1], we set this limit to 10 min. All computations have been done on a computer equipped with a Intel Xeon CPU, 3.20GHz, 64GB of RAM.

Previous exact algorithms. In addition to the basic Branch and Bound **BAB** of Solano [14], we are aware of the following implementations of algorithms to compute vertex-separation of graphs.

MILP. Some mixed integer linear programming formulations for the vertex-separation (MILP) have been proposed in [1, 14]. A similar formulation has been implemented in *Sage*² and it uses the CPLEX solver. We use this function for purpose of comparison.

DYNPROG. This is a dynamic programming algorithm that computes the vertex-separation of a digraph D [2]. An implementation of DYNPROG is available in *Sage*². Since the worst case time and space complexities of DYNPROG is $O(2^n)$, it can be used only for graphs with strictly less than 32 nodes.

SAT. In [1], an algorithm to compute the pathwidth of graphs is designed by formulating the PATHWIDTH problem as a Boolean satisfiability testing (SAT) instance and solving this SAT instance using the *MiniSat* solver. No implementations are provided but the algorithm has been tested on the *Rome graphs* dataset [12]. The SAT algorithm have been able to compute the pathwidth of 17% of the instances in the *Rome graphs* dataset.

4.2 Evaluation in Random Digraphs

In this section, we use the available implementations of MILP and DYNPROG in *Sage* to compare their performance with our algorithms in random directed graphs. Random directed graphs with N nodes are generated using the `graphs.RandomDirectedGNM(N,M)` method of *Sage* to generate them, where $M = \text{density} * N(N - 1)$ is the number of arcs. For several network sizes ($N \in \{20, 25, 30, 40, 50\}$), we execute the algorithms for various densities ($0 < \text{density} < 1$). For any N and each density, we run the algorithms on 1000 instances. The same instances are used for all algorithms. The average running times are depicted on Fig. 1.

Small Graphs ($N = 20$). Fig. 1(a) shows that the MILP Algorithm is outperformed by all other algorithms even for $N = 20$ (two weeks of computations have been required to generate Fig. 1(a) because of MILP). Fig. 1(a) also shows that, for $N = 20$, Algorithm DYNPROG is slightly faster than all variants of

² Module `sage.graphs.graph_decompositions.vertex_separation`

BAB. In particular, the running time of **BAB-GPP** is a bit larger than for other variants. This is probably due to the fact that the cuts on the exploration space done by the optimization procedures executed by **BAB-GPP** are negligible compared with the time needed to apply the optimization procedures.

Graphs with $N \in \{25, 30\}$. Figs. 1(b) and 1(c) show that, for graphs with at least 25 nodes, our algorithms become competitive with respect to DYNPROG. In particular, for graphs with 30 nodes, our algorithms are significantly faster (recall that DYNPROG can only be used for graphs with at most 31 nodes).

For $N \geq 30$, the basic variant of **BAB** behaves as well as **BAB-G** for *density* ≥ 0.5 , but for smaller densities the computation time is not competitive (several hours). Therefore, we did not report on its performances on the plots.

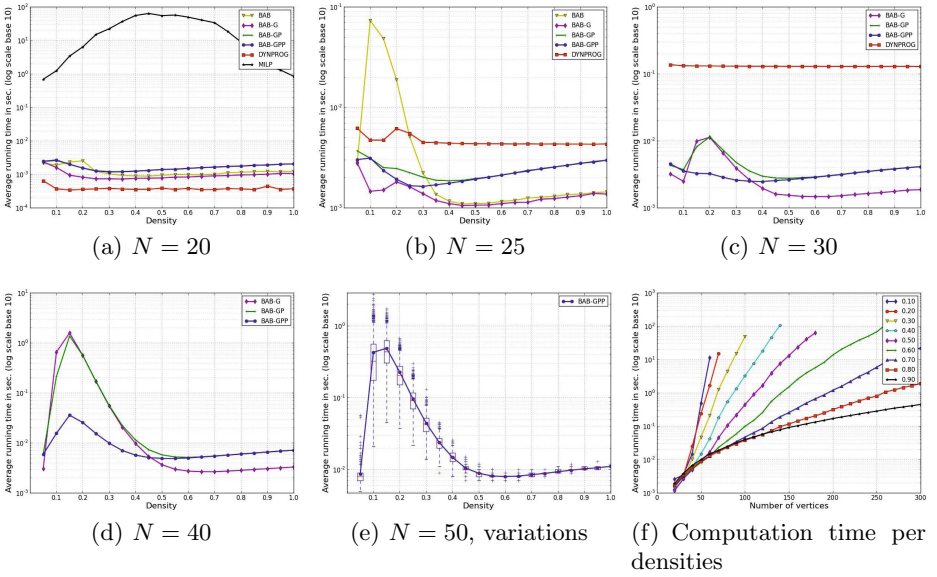


Fig. 1. Average running time of the algorithms on random digraphs

Impact of Optimization Phases. In Figs. 1(b) to 1(e), we observe that for densities ≥ 0.4 , all variants of the Branch and Bound algorithm are very fast with negligible differences (≤ 0.01 sec.). However, we observe large variations for smaller densities. More precisely, we observe in Fig. 1(b) the significant speed-up offered by the Greedy steps w.r.t. the basic Branch and Bound proposed in [14]. The benefits of the pre-processing phases are however difficult to observe on such small random digraphs since the conditions required to contract arcs rarely occur, and so we observe an improvement in Fig. 1(d) only when the density is 0.05. However, Fig. 1(d) reports large speed-up when using prefixes to

cut the search space. In Fig. 1(e), we point out that the running time of **BAB-GPP** varies a lot depending on the graphs (especially for graphs with small density). We report on the variations in running time when $N = 50$ and using the best settings of our algorithm. For densities below than 0.4, the running time varies by up to two orders of magnitude, while for larger densities, the range of variations is very small. Last, we have reported in Fig. 1(f) the evolution of the running time of **BAB-GPP** for different densities. This confirms that the higher the density, the larger the size of the graphs we are able to solve. Note that we have observed the same behaviors for all algorithms with undirected graphs.

4.3 Comparison of BAB-GPP with SAT on *Rome Graphs*

The *Rome graphs* dataset [12] consists of 11 529 undirected n -node graphs with $10 \leq n \leq 100$. In [1], the performance of the SAT algorithm has been evaluated using this benchmark³. With 10 min. time limit per graph, the algorithm of [1] has been able to compute the pathwidth for 17.0% of the Rome graphs. In particular, it is stated that “We note that almost all small graphs ($n + m < 45$) could be solved within the given timeout, however, for larger graphs, the percentage of solved instances rapidly drops [...] Almost no graphs with $n + m > 70$ were solved.” [1] (where m is the number of edges of the considered graphs).

In contrast, our algorithm has computed the pathwidth of 95.6% of the graphs in the *Rome dataset*, with same time limit of 10 min. Note that, in particular, we solved all instances with at most 82 vertices. We report in Tab. 1 the repartition of (un)solved instances.

Table 1. Repartition of (un)solved instances

N	≤ 82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	Total
Nb graphs	9586	86	78	73	70	68	69	63	59	116	119	134	154	139	148	139	143	144	141	11 529
Solved	9586	85	77	68	65	62	65	59	50	96	91	104	113	91	110	80	77	70	78	11 027
Unsolved	0	1	1	5	5	6	4	4	9	20	28	30	41	48	38	59	66	74	63	502

	Max. prefix length	Time (in sec.)	Visited nodes	Stored prefix
	1	2226.45	1 256 780 074	44
	5	144.93	82 417 700	16 386
	8	5.81	3 319 482	47 756
	10	1.07	664 677	61 466
k	1 2 3 4 5 6	7 8		
$ V(M_k) $	1 2 5 11 23 47	95 191		
$pw(M_k)$	0 1 2 5 10 20	$\leq 38 \leq 72$		
		15	496 482	65 252
		20	496 482	65 253

Fig. 2. Pathwidth of some Mycielski graphs

Fig. 3. Running time of **BAB-GPP** for the Mycielski graph M_6

³ Computations in [1] have been performed on a computer equipped with a AMD Opteron 6172 processor operating at 2.1GHz and 256GB of RAM.

4.4 Impact of Prefix Length

Our main algorithm **BAB-GPP** is parameterized by the maximum length of the prefixes that we store and that allow us to cut the search space during the Branch and Bound process. We have evaluated the impact of this parameter when our algorithm is executed on some specific graphs. In particular, the *Mycielski graph* is considered as a hard instance for integer programming formulations for pathwidth [11]. The Mycielski graph M_k is a triangle-free graph with chromatic number k having the smallest possible number of vertices. Note that, $|V(M_2)| = 2$ and $|V(M_k)| = 2|V(M_{k-1})| + 1$ for any $k > 2$. We ran our algorithm on the Mycielski graph M_k for $k \leq 8$. Our results are depicted in Tab. 2 (for $k \in \{7, 8\}$ we only got upper bounds).

Tab. 3 presents the running time of **BAB-GPP** for the Mycielski graph M_6 and different bounds on the size of the stored prefixes. In Tab. 3, the column *Visited nodes* refers to the number of nodes of the Branch and Bound exploration space that are actually considered. Tab. 3 shows that storing larger prefixes allows for an impressive reduction of the computation time. However, we have no improvement here when allowing length of 20 instead of 15, and larger values behave similarly. This is probably due to the fact that in this experiment, because of the greedy steps, the algorithm stores only one prefix of length ≥ 15 .

This suggests that the combination of the greedy steps and of the storage of the prefixes allows good performance even with bounded length of the prefixes. Thus, memory-space seems not to be an issue for graphs of reasonable size (even though the algorithm might potentially store an exponential number of prefixes).

4.5 Using BAB-GPP as an Heuristic

Some research has been devoted to design heuristic algorithms for this problem. [9] proposed the following: starting from some layout of the nodes of D , the solution is improved by switching pairs of nodes in the layout until no further improvement can be obtained. The VSPLIB [16] has been designed for benchmarking this heuristic [9]. VSPLIB [16] contains 173 instances: 50 $\gamma \times \gamma$ grids with $5 \leq \gamma \leq 54$; 50 trees with respectively 22, 67, and 202 nodes and pathwidth 3, 4, and 5; a set of 73 graphs of order n , called HB, with $10 \leq n \leq 960$ (see [9] for more details). In particular, the pathwidth of $\gamma \times \gamma$ grids equals γ and the pathwidth of trees can be computed in linear time [8]. Therefore, this benchmark allows to evaluate the performance of heuristic. We tested the Algorithm **BAB-GPP** as a heuristic on the graphs of VSPLIB. If the time is up before the end of **BAB-GPP**'s execution, the algorithm returns the value computed so far (i.e., an upper bound on the pathwidth of the graph).

We were able to compute the exact pathwidth for all grids with side $\gamma \leq 13$, trees with $n \leq 67$, and 26 of the HB graphs including one with 957 nodes. For all grids and trees of the VSPLIB, the final value returned by **BAB-GPP** equals the exact pathwidth. That is, our algorithm always finds quickly an optimal layout and most of the execution time is devoted to prove its optimality.

In particular for a $\gamma \times \gamma$ grid, the first solution found is always the pathwidth. This is due to the order in which we add the vertices in the layouts: starting

from a node with smallest degree and always adding a node that minimizes the increase of the size of the border. Proceeding that way will always give an optimal layout in square grids.

For trees, the first layout tested by **BAB-GPP** is not always the optimal one, but an optimal one is found very quickly. It would be interesting to understand why our Branch and Bound algorithm performs well in trees⁴.

5 Conclusion

In this paper, we have presented a new Branch and Bound algorithm for computing the pathwidth of graphs and the vertex-separation of digraphs. Our approach is more promising than previous proposals, based on ILP or SAT, for solving large instances. Indeed, the drawbacks of ILP and SAT formulations for layout problems are both in the large number of symmetries of the problems, and in the time needed to fill the optimality gap (i.e., distance between the lower bound based on the fractional relaxation of the formulation and the best integral solution). Our next steps are on one hand to propose our code for inclusion into future releases of *Sage*. On the other hand, we will search for new pruning rules to further reduce computation time. In particular, looking for good lower bounds for pathwidth of graphs is a theoretical issue that received few attention. It would moreover speed up our algorithm since the main computation time seems dedicated to prove the optimality of the best value computed.

References

1. Biedl, T.C., Bläsius, T., Niedermann, B., Nöllenburg, M., Prutkin, R., Rutter, I.: Using ILP/SAT to determine pathwidth, visibility representations, and other grid-based graph drawings. In: Wismath, S., Wolff, A. (eds.) GD 2013. LNCS, vol. 8242, pp. 460–471. Springer, Heidelberg (2013)
2. Bodlaender, H.L., Fomin, F.V., Koster, A.M., Kratsch, D., Thilikos, D.M.: A note on exact algorithms for vertex ordering problems on graphs. *Theory Comput. Syst.* 50(3), 420–432 (2012)
3. Bodlaender, H.L., Jansen, B.M.P., Kratsch, S.: Kernel bounds for structural parameterizations of pathwidth. In: Fomin, F.V., Kaski, P. (eds.) SWAT 2012. LNCS, vol. 7357, pp. 352–363. Springer, Heidelberg (2012)
4. Bodlaender, H.L., Kloks, T.: Efficient and constructive algorithms for the pathwidth and treewidth of graphs. *J. Algorithms* 21(2), 358–402 (1996)
5. Bodlaender, H.L., Koster, A.M.C.A.: Treewidth computations ii. lower bounds. *Inf. Comput.* 209(7), 1103–1119 (2011)
6. Coudert, D., Huc, F., Mazauric, D., Nisse, N., Sereni, J.-S.: Reconfiguration of the routing in WDM networks with two classes of services. In: *Optical Network Design and Modeling (ONDM)*, pp. 1–6. IEEE (2009)
7. Coudert, D., Mazauric, D., Nisse, N.: Experimental evaluation of a branch and bound algorithm for computing pathwidth. Tech. Rep. RR-8470. Inria (February 2014)

⁴ Note that optimal path-decompositions are computable in linear time in trees [13].

8. Díaz, J., Petit, J., Serna, M.: A survey on graph layout problems. *ACM Comput. Surveys* 34(3), 313–356 (2002)
9. Duarte, A., Escudero, L.F., Martí, R., Mladenovic, N., Pantrigo, J.J., Sánchez-Oro, J.: Variable neighborhood search for the vertex separation problem. *Computers & OR* 39(12), 3247–3255 (2012)
10. Hein, A., Koster, A.M.C.A.: An experimental evaluation of treewidth at most four reductions. In: Pardalos, P.M., Rebennack, S. (eds.) *SEA 2011*. LNCS, vol. 6630, pp. 218–229. Springer, Heidelberg (2011)
11. MIPLIB - mixed integer problem library, <http://miplib.zib.de/>.
12. Rome graphs, <http://www.graphdrawing.org/download/rome-graphml.tgz>
13. Skodinis, K.: Construction of linear tree-layouts which are optimal with respect to vertex separation in linear time. *J. Algorithms* 47(1), 40–59 (2003)
14. Solano, F., Pióro, M.: Lightpath reconfiguration in WDM networks. *IEEE/OSA J. Opt. Commun. Netw.* 2(12), 1010–1021 (2010)
15. Stein, W., et al.: Sage Mathematics Software (Version 6.0). The Sage Development Team (2013), <http://www.sagemath.org>
16. VSPLIB (2012), <http://www.opticom.es/vsp/>